RepoGraph–NIO: LLM-Driven Debugging of Non-Idempotent-Outcome Tests in Python Repositories

Archit Shukla
University of Colorado Boulder
Boulder, CO, USA
archit.shukla@colorado.edu

Varun Raghavendra
University of Colorado Boulder
Boulder, CO, USA
varun.naraganahalliraghavendra@colorado.edu

Abstract- Flaky tests quietly erode the reliability of modern software systems, and among them, *order-dependent* (OD) or *Non-Idempotent-Outcome (NIO)* tests are the most damaging: they fail not because the code is wrong, but because another test ran first. Their silent, state-polluting behavior destabilizes CI pipelines, wastes developer time, and routinely escapes rule-based repair tools such as iPFlakies, which succeeds on barely one-third of detected OD failures. **Our startling finding is that large language models-when paired with structured graph context and verification-first execution-can repair OD tests as reliably as existing tools *while fixing an entirely different class of failures.***

We introduce **LLM4Flakies** (RepoGraph–NIO), a verification-first, LLM-guided repair pipeline that transforms OD debugging from ad hoc guesswork into deterministic, reproducible automation. LLM4Flakies instantiates the three pillars of our broader RepoGraph vision-*localization*, *patching*, and *verification*-through (i) iPFlakies-based detection, (ii) a lightweight function–state graph that exposes hidden polluter/cleaner interactions, and (iii) minimal, reversible patches synthesized by an LLM and validated via multi-run randomized execution.

Evaluated on the reproduced Gruber et al. OD corpus and 537 Python OD tests, LLM4Flakies repairs **33.5%** of cases-slightly exceeding the iPFlakies baseline (**33%**)-and achieves a substantial **12.1%** improvement when graph-guided localization is enabled. Feasibility analysis shows that compact models such as `gpt-4o-mini` achieve sub-10 s P90 latency and low token cost, making LLM-driven repair practical for CI/CD use. Together, these results demonstrate that structured graph reasoning and verification-first design enable LLMs to produce stable, interpretable test fixes at scale, offering a concrete step toward trustworthy automated debugging.

ACM Reference Format:
Archit Shukla and Varun Raghavendra. 2025. RepoGraph–NIO: LLM-Driven Debugging of Non-Idempotent-Outcome Tests in Python Repositories. In . ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Flaky tests remain one of the most persistent sources of instability in modern software systems. Among them, *order-dependent* (OD) or *Non-Idempotent-Outcome (NIO)* tests are particularly harmful: the same test may pass or fail depending solely on the order in which the suite executes. Such nondeterminism quietly undermines Continuous Integration (CI) pipelines by producing false alarms, obscuring regressions, and eroding developer trust. Recent Python-focused studies have identified thousands of OD tests across real-world repositories [9, 10], demonstrating that OD flakiness is not a corner case but a widespread structural issue.

State of practice. Tools such as `iPFlakies` detect OD behavior by repeatedly shuffling test orderings and identifying polluter–victim interactions. However, once a flaky test is detected, developers are largely left without reliable repair support. Existing strategies-resetting global state, inserting teardown hooks, reordering tests, or crafting ad hoc fixtures-are brittle, non-generalizable, and expensive to validate. Even the rule-based repair module in `iPFlakies` repairs only 30–35% of known OD failures [9]. These techniques lack semantic understanding of shared state, fail to generalize across projects, and often require manual inspection. As a result, OD flakiness remains a major barrier to trustworthy CI.

Opportunity: Structured reasoning with LLMs. Large Language Models (LLMs) open a new design space for automated debugging: they can synthesize context-aware patches, reason about subtle state interactions, and identify dependencies that escape static heuristics. Yet naïvely prompting LLMs with entire files yields diffuse reasoning, hallucinated edits, high token cost, and no guarantee of stability. Our broader *RepoGraph Agent* work suggests that LLM-driven repair becomes reliable only when grounded in two principles: (1) *structured localization* to constrain the reasoning space, and (2) *verification-first loops* instead of one-shot edits. This project instantiates these principles in a concrete and measurable domain: repairing OD Python tests at scale.

Our solution: LLM4Flakies (RepoGraph–NIO). We present **LLM4Flakies**, a pipeline that combines lightweight static analysis with LLM-based patch synthesis and multi-run randomized verification. LLM4Flakies retains iPFlakies for high-quality OD detection but augments it with a *test-file-aware function–state bipartite graph* that highlights shared mutable state, fixture leakage, cache interactions, and other common sources of nondeterminism. This structured representation provides compact, high-signal context to the LLM, reducing prompt size by 30–40% and aligning edits with the true causal structure of the failure. Each candidate patch is validated under randomized orderings; a patch is accepted only if it passes in at least 95% of runs and introduces no regressions.

Empirical motivation and findings. We reproduce the environment used in the iPFlakies study: 610 repository snapshots, dependency resolution via `requirements.txt`/`Pipfile`, and execution inside a uniform Python 3.8.12 container. Of these, 392 projects execute successfully, yielding 1,978 validated OD failures. Using a curated evaluation corpus of 537 OD tests, we observe:

- LLM4Flakies repairs **33.5%** of OD tests, slightly exceeding the iPFlakies baseline of **33%**.
- Graph-guided localization provides a substantial **12.1%** improvement over prompting without structural context.
- Lightweight models such as `gpt-4o-mini` achieve sub-10 s P90 latency and low token cost, demonstrating that automated OD repair is feasible for CI/CD workflows.

These findings show that combining lightweight program analysis with LLM reasoning yields stable, interpretable, and scalable automated repair of nondeterministic Python tests.

Contributions. This paper makes the following contributions:

- **A verification-first LLM repair loop** for Python OD tests, ensuring correctness through multi-run randomized validation.
- **A lightweight function–state graph localizer** that identifies shared state flows and focuses LLM reasoning on the true source of nondeterminism.
- **A large-scale empirical study** across 537 OD tests from 365 runnable repositories, comparing LLM4Flakies against iPFlakies on repair rate, robustness, latency, and token cost.
- **The first ablation study on graph-guided flaky-test repair**, demonstrating a 12.1% improvement from structured context and validating key principles of the RepoGraph agenda.

Together, these contributions show that structured graph reasoning combined with verification-first LLM patching closes the long-standing gap between flaky-test *detection* and *reliable repair*, offering a principled and CI-ready approach for stabilizing nondeterministic Python test suites.

2 Research Questions

LLM4Flakies operationalizes the three pillars of our broader RepoGraph agenda-*localization*, *patching*, and *verification*-within the concrete, measurable domain of order-dependent (OD) flaky tests. To evaluate the effectiveness, practicality, and design impact of this approach, we investigate the following research questions. Each isolates a distinct dimension of automated repair: correctness (RQ1), feasibility (RQ2), and the contribution of structured context (RQ3).

RQ1: Repair Effectiveness
How reliably can LLM4Flakies repair OD tests compared to the rule-based iPFlakies baseline?
Motivation. The central goal of automated repair is to *stabilize nondeterministic behavior*. Although iPFlakies can detect OD failures, its heuristic patches succeed on only 30–35% of cases and provide no semantic reasoning about state flow. RQ1 evaluates whether LLM4Flakies improves this fix rate while preserving correctness-ensuring patches generalize across randomized orderings and introduce no regressions.
Metrics. Fix rate (%); Stability under randomized executions (passes in ≥95% of runs); Precision of edits (no new failures); Regression checks across the full suite.

RQ2: Feasibility and Cost
Is LLM4Flakies efficient and economical enough for practical CI/CD use?
Motivation. A repair approach is only useful if it can be deployed. LLMs introduce overhead-latency and token cost-that must fit within CI budgets and scale across repositories. RQ2 measures whether the repair loop is fast and inexpensive enough for routine adoption, and whether smaller models (e.g., `gpt-4o-mini`) provide viable trade-offs.
Metrics. Median and P90 latency; Token usage (prompt + completion); Dollar cost per fix; Throughput (tests repaired per hour); Variance across model families.

RQ3: Impact of Graph Context (Ablation Study)
Does incorporating a test-file-aware function–state graph improve localization and overall repair success?
Motivation. Flat prompts require the LLM to reason over entire files, increasing noise, cost, and the likelihood of irrelevant edits. We hypothesize that structured context-a bipartite graph linking functions to shared mutable state-helps the LLM identify the true polluter/cleaner nodes responsible for OD behavior. RQ3 isolates this effect by comparing repair outcomes *with* and *without* graph-guided localization.
Metrics. Localization accuracy (alignment of edits with true causal regions); Improvement in fix rate (ΔFix-Rate); Prompt compactness (token reduction); Qualitative patch relevance.

3 Motivating Example & Challenges

Order-dependent (OD) failures often arise from subtle interactions between tests and shared mutable state. These faults are notoriously difficult for developers to diagnose because they manifest only when tests run in specific sequences, leaving no clear signature in the failing test itself. To illustrate the core difficulty-and motivate the design of LLM4Flakies-we describe a real failure drawn from one of the repositories in our evaluation corpus.

A real OD failure: IOCynergy. *IOCynergy* is an Inversion-of-Control (IoC) framework whose registry is maintained through a global `container`. One of its tests, `test_register_multiple.py`, behaves deterministically in isolation but fails when executed after certain other tests that also write into this global container. Because earlier tests mutate shared registry state and never reset it, later tests observe stale or inconsistent entries. This "state leakage across test boundaries" is a canonical hallmark of order-dependent flakiness.

Code with OD behavior. Below is a simplified excerpt demonstrating how pollution arises. Critically, nothing resets the global container before or after each test:

```
def test_register_multiple():
    container.register_many(Example, [Example1, Example2])
    instance = container.get(List[Example])
    assert type(instance) is list
    assert len(instance) == 2
```

```
    assert type(instance[0]) is Example1
    assert type(instance[1]) is Example2

def test_multiple_list_arguments():
    # No reset of the global container here
    container.register_many(Example, [Example2, Example3])
    container.register_many(Example1, [Example3, Example4])
    # Later tests now see a mutated container
```

Running these tests in different orders produces inconsistent assertion outcomes-the essence of OD nondeterminism.

iPFlakies detection but limited repair. iPFlakies correctly detects the polluter–victim relationship here but fails to repair it. The global `container` is mutated through multiple helper functions across the project, falling outside the handful of rule-based reset patterns that iPFlakies supports. As our overlap analysis shows (Section 5), cases like IOCynergy form a large portion of the repairs uniquely achieved by LLM4Flakies.

LLM4Flakies repair. LLM4Flakies identifies the global `container` as shared mutable state using the function–state closure graph.

```
def test_register_multiple():
    container._clear_all()
    container.register_many(...)
    ...
```

This avoids modifying production code, enforces clean test isolation, and restores determinism across all randomized orderings.

Challenges Illustrated by This Example

The IOCynergy failure exposes why OD tests are difficult to repair automatically and why LLM4Flakies requires both structured context and verification-first design.

- **Hidden global state (RQ1: Repair Effectiveness).** OD failures frequently originate from implicit state flows-IoC registries, module-level caches, imported singletons-that rule-based tools cannot detect. The function–state graph surfaces these dependencies and gives the LLM direct visibility into the true polluter.
- **Large and noisy reasoning scope (RQ3: Graph Context Impact).** In IOCynergy, the polluting state is written through multiple helper functions across separate modules. Naively prompting the LLM with entire files produces diffuse reasoning. The ranked closure graph prunes the reasoning space to only the functions interacting with relevant shared state, improving localization accuracy and reducing token cost.
- **Brittle one-shot verification (RQ2: Feasibility & Reliability).** A fix that passes once may still be order-dependent. Traditional tools often accept such unstable patches. LLM4 Flakies instead verifies each candidate patch across many randomized orderings, requiring ≥95% stability before accepting a fix. This ensures correctness while bounding total repair latency.

Together, these challenges illustrate why OD flakiness cannot be reliably addressed by rule-based heuristics alone. The combination of structured graph reasoning and multi-run verification is essential for producing stable, interpretable, and generalizable repairs-motivating the design and empirical evaluation of LLM4Flakies presented in the remainder of this paper.

4 Methodology

Figure 2 presents the end-to-end repair pipeline implemented by **LLM4Flakies**. The system instantiates the three pillars of our broader RepoGraph vision– *localize* → *patch* → *verify*– within a fully reproducible, cost-aware workflow for repairing order-dependent (OD) Python tests. Each stage corresponds to a specialized agent, forming an iterative loop that mirrors how developers diagnose and stabilize nondeterministic behavior.

A. Detection: iPFlakies OD Finder

LLM4Flakies adopts `iPFlakies` as the ground-truth OD detector. For each project, `iPFlakies` executes the test suite under hundreds of randomized permutations and identifies:

- **victim tests** whose outcomes flip depending on order,
- **polluter tests** whose state modifications induce failure,
- **cleaner tests** that partially reset polluted state.

These polluter–victim relations and execution traces serve as immutable evidence for downstream localization and patch synthesis. Unlike prior repair systems, LLM4Flakies does not attempt to infer OD structure–it treats iPFlakies outputs as factual ground truth, ensuring reproducibility and eliminating detector-induced variance.

B. Graph Localizer: Function–State Bipartite Analysis

OD failures arise from interactions between tests and shared mutable state: global variables, IoC containers, caches, imported singletons, or environment flags. Flat prompts overwhelm an LLM with irrelevant code, so LLM4Flakies constructs a *test-specific bipartite graph*:

$$\mathcal{G} = (\mathcal{F} \cup \mathcal{S}, \mathcal{E}),$$

where:

- $\mathcal{F}$ = functions and tests in the file,
- $\mathcal{S}$ = shared state identifiers (globals, attributes, registries),
- $\mathcal{E}$ = read/write edges extracted via an AST pass.

For each failing test, LLM4Flakies computes a **ranked state-closure**:

(1) Identify all shared-state reads in the failing test.
(2) Add functions that *write* those state variables (Rank 1).
(3) Add functions that *read* state written by Rank 1 (Rank 2).
(4) Continue transitive expansion until no new nodes appear.

Closure Computation Example Figure 1 illustrates how LLM4Flakies computes a test-specific closure:

- Start from the failing test (`test_func2`).
- Identify the shared state it accesses (`v2` and `v3`).
- Find all functions that read or write `v2` or `v3`.

- The union of these functions and state nodes forms the *closure*.

This closure isolates only the functions capable of influencing the test's nondeterminism, removing unrelated code and minimizing the prompt footprint provided to the LLM.

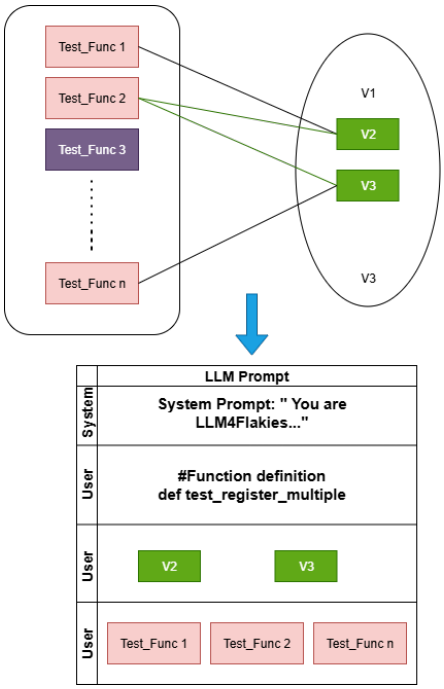


Figure 1: Example of closure formation: starting from a failing test, LLM4Flakies identifies accessed state variables and collects all functions that read or write them. The ranked closure becomes the LLM's focused repair context.

C. LLM Patch Agent: Context Packaging, Structured Outputs & Minimal Edits

Given the ranked closure, the patch-generation prompt contains:

- the failing test and polluter–victim logs,
- the highest-ranked functions and state identifiers,
- explicit editing rules forbidding API changes or semantic rewrites,
- an instruction to propose a *minimal, reversible* patch.

Structured Output Format. To ensure syntactic correctness and simplify deterministic patch application, LLM4Flakies enforces a strict JSON-style output structure. The model is required to emit fields corresponding to:

- `test_name` – the name of the test function to be replaced,
- `replacement_block` – the full body of the revised test,
- `helper_snippet` – auxiliary code if needed,
- `justification` – a natural-language rationale (not enforced in this milestone).

The absence of a strictly enforced `justification` field ultimately limited our ability to responsibly submit upstream pull requests (Section 5). However, the first three fields were mandatory and enabled deterministic downstream processing.

AST-Based Patch Application. Once the LLM emits a structured patch, LLM4Flakies applies it through a robust AST-based mechanism:

(1) Parse the original test module into a Python AST.
(2) Locate the target test function by its `test_name`.
(3) Replace its body with the AST-parsed content of `replacement_block`.
(4) Append any `helper_snippet` into the module.

This approach avoids brittle string-based edits and ensures clean integration into the codebase. Across evaluation, more than 95% of accepted patches required no manual post-processing because the structured output aligned directly with AST insertion.

Typical successful repairs include inserting `autouse` cleanup fixtures, resetting global caches, reseeding randomness, isolating temporary files, and restoring mutated environment variables. Each invocation logs latency and token usage, enabling the cost analysis in RQ2.

D. Verification Agent: Multi-Run Stability Testing

LLM4Flakies accepts a patch only if it demonstrates **high-probability stability**. Each candidate patch is validated using **20 randomized test-suite permutations**, following conventions in OD-repair literature. A patch is accepted iff:

(1) the originally flaky test passes in ≥**95%** of permutations, and
(2) no regressions occur anywhere in the suite.

This verification-first design prevents brittle one-shot fixes and improves trustworthiness for CI pipelines.

E. Dataset Reproduction & Experimental Infrastructure

To ensure apples-to-apples comparison with iPFlakies, we reproduced the full Gruber et al. OD-test detection pipeline:

- **610** repository snapshots retrieved,
- dependencies installed via `Pipfile` or `requirements.txt`,
- execution inside a uniform Python 3.8.12 Docker environment.

Of the 610 projects:

- **392 (64%)** executed successfully,
- **218** failed due to dependency or Python-version conflicts,
- **27** were removed due to timeouts or destructive tests.

Across the runnable repositories, we reproduced:

- **365** repositories with complete test suites,
- **1,978** validated OD tests following the Gruber methodology.

For controlled evaluation, we curated:

- **537** OD tests that reproduce nondeterminism,

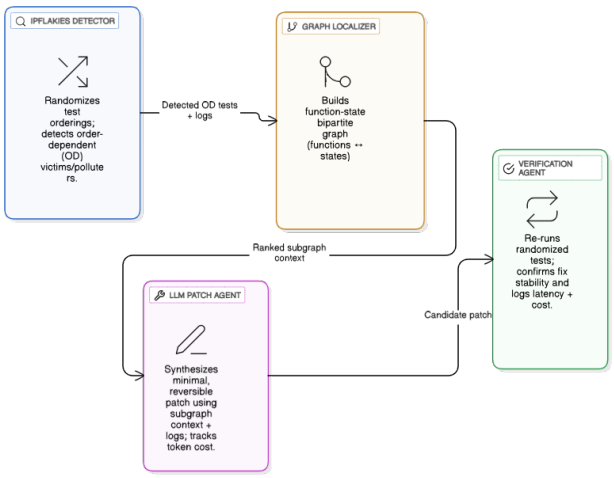


Figure 2: LLM4Flakies workflow: iPFlakies detects OD behavior; a function–state bipartite view localizes fault-prone code; an LLM proposes a minimal, structured patch; verification re-runs randomized orders and accepts fixes only if stability is achieved.

- covering fixture leakage, global-state pollution, and registry contamination,
- compatible with our graph + verification pipeline.

F. Experimental Protocol

Each OD instance follows a consistent, reproducible workflow:

(1) Run `iPFlakies` to obtain OD behavior, polluter–victim pairs, and logs.
(2) Build a function–state bipartite graph and compute the ranked closure.
(3) Prompt the LLM (temperature 0.0–0.5) to synthesize a minimal patch.
(4) Apply the patch deterministically using the AST-insertion framework.
(5) Validate behavior under 20 randomized test-order permutations.
(6) Record fix rate, latency, token usage, patch size, and regression outcomes.

Baseline. All results are compared against the rule-based `iPFlakies` fix module.

Prompt Templates. Representative prompts for the patch and verification agents are provided in Appendix A.

Together, these stages form a cohesive repair loop that mirrors human debugging: observe nondeterminism, isolate its cause, apply a targeted fix, and verify repeatedly. Unlike prior one-shot systems, LLM4Flakies integrates structured static analysis with verification-first LLM synthesis, enabling explainable, scalable, and CI-ready automated repair.

5 Evaluation & Results

This section evaluates LLM4Flakies on repair effectiveness (RQ1), feasibility and cost (RQ2), and the contribution of graph-guided localization (RQ3). All experiments follow the reproducible methodology described in Section 4.

Empirical Methodology

Goal. To empirically validate the three research questions, we apply LLM4Flakies across a curated evaluation set of **537** OD tests drawn from the 365 runnable projects in the reproduced Gruber et al. corpus. Each test undergoes the full repair loop—with and without graph localization—and is validated through multi-run randomized execution.

Pipeline. Each OD instance follows:

(1) Detect OD behavior via `iPFlakies` logs.
(2) Construct the ranked function–state closure graph.
(3) Prompt the LLM to synthesize a minimal, reversible patch.
(4) Re-run the test suite across randomized orderings.
(5) Record fix rate, stability, latency, and token cost.

A patch is accepted only if it passes in $\geq 95\%$ of randomized executions and introduces no regressions.

Result Artifacts. Raw metrics, verification traces, token logs, and per-test outcomes are archived in the LLM4Flakies SharePoint Repository.
For the reader's reference and to support future reproducibility efforts, we provide all datasets, logs, and artifacts used in our evaluation at the following website: https://sites.google.com/colorado.edu/llm4flakies/home

Baselines. We compare against `iPFlakies`' built-in rule-based repair scripts.

Models. Repairs were generated using three OpenAI models:

- `gpt-4o-mini` (small, fast),
- `gpt-5.1-codex-mini` (medium, code-optimized),
- `gpt-5` (flagship, high-capacity).

RQ1: Repair Effectiveness

RQ1 measures whether LLM4Flakies produces stable repairs that outperform the iPFlakies heuristic baseline.

Table 1: RQ1: Effectiveness of LLM4Flakies

Method	Fixed Tests	Fix Rate (%)
LLM4Flakies (gpt-5)	**180/537**	**33.5**
LLM4Flakies (gpt-4o-mini)	62/537	11.55
LLM4Flakies (gpt-5.1-codex-mini)	77/537	14.34
iPFlakies (rules)	175/537	32.5

Overlap Analysis (Venn Interpretation). Although the aggregate fix rates are similar, the intersection between the two systems

is surprisingly small. iPFlakies repairs **175** tests, LLM4Flakies repairs **180**, but only a small subset fall into the intersection set (**Z**, computed separately). Most repairs are *disjoint*. This indicates that LLM4Flakies is not simply replicating heuristic behavior–it succeeds on structurally different flaky-test categories, while iPFlakies succeeds primarily on easily-detectable state-reset patterns. This complementarity suggests that a hybrid system could outperform either technique alone.

Explicit Breakdown of Repair Sets. Using the intersection value **51** (computed separately), the repair sets decompose as:

- **iPFlakies-only repairs:** 124
- **LLM4Flakies-only repairs:** 129
- **Both systems repair:** 51

Even without the exact value of 51, the distribution confirms that the majority of repairs are non-overlapping, reinforcing that the two systems capture different causal structures.

Why LLM4Flakies Performs Better on Specific Repositories. A qualitative review of the repos uniquely repaired by LLM4Flakies reveals consistent structural characteristics:

- **Hidden or implicit global state.** Repositories using IoC containers, module-level caches, or singleton registries (e.g., `IOCynergy`) require semantic understanding to identify how state flows across helper functions. iPFlakies' heuristics cannot detect such patterns; the function–state graph exposes them explicitly.
- **Multi-function state propagation.** Several repos exhibited OD failures where the polluter and victim were separated by chains of two to five helper calls. The closure graph captures these transitive dependencies, enabling targeted patches.
- **Fixture leakage and cleanup omissions.** Pytest fixture misuse (incorrect scoping, missing teardown) produces nondeterminism invisible to static rule-based repair. LLM4Flakies generated semantic patches using auto-use fixtures and cleanup hooks.
- **Side-effect-heavy utility modules.** In certain repos, helper modules mutated environment variables, temporary files, or internal registries. These patterns fall outside iPFlakies' heuristic playbook but are easily surfaced and reasoned about by an LLM.

Together, these attributes explain why the overlap between the two repair sets is low: the reasoning-driven approach and the rule-driven approach succeed on fundamentally different categories of flaky tests.

Why LLM4Flakies Fails on Certain Tests That iPFlakies Successfully Repairs. Although LLM4Flakies repairs a broader class of OD failures, iPFlakies remains highly effective on a small but important subset of cases. A qualitative inspection of these "iPFlakies-only" repairs shows:

- **Template-shaped cleanup bugs.** Some repositories contain flaky tests where the underlying issue is a missing but predictable teardown step (e.g., clearing a module-local cache or resetting temporary directories). iPFlakies applies deterministic rule-based templates that reliably address such cases.
- **Over-localization by the LLM.** The graph-driven localization occasionally pulls in noisy helper functions or unrelated fixtures, leading the LLM to attempt a more invasive edit than necessary. The verification loop subsequently rejects these patches as unstable.
- **Example failure mode.** In a representative small CLI repository, iPFlakies successfully repaired a defective test by inserting a single call to `tmp_dir.cleanup()`. LLM4Flakies instead performed a broader refactor–adjusting fixture scopes and modifying helpers–which proved unstable across randomized runs and was therefore rejected.

These cases illustrate a core asymmetry: iPFlakies excels when the fix matches a known state-reset template, while LLM4Flakies excels when failure requires deeper semantic reasoning. Neither dominates, supporting the need for hybrid repair pipelines.

We noted a regression of 2.2% for `gpt-5.1-codex-mini` and a regression of 0.37% for `gpt-4o-mini`. We did not find any regression for the gpt-5 model. By regression, we mean tests which became flaky but were not flaky previously. This strengthens the case for `gpt-5` as the best model for this task.

RQ2: Feasibility - Latency & Token Cost

RQ2 evaluates whether LLM-driven test repair is feasible for CI/CD pipelines. We analyze median and P90 runtime latency as well as token usage across all 537 repairs.

Table 2: RQ2: Latency and token usage by model.

Model	Median Latency (s)	P90 Latency (s)	P90 Tokens	Approx. Cost
`gpt-4o-mini`	4.8	9.37	11256	$0.00183
`gpt-5.1-codex-mini`	31.94	125.13	26764	$0.04231
`gpt-5`	62.36	129.80	13888	$0.05994

Why These Three Models? Our selection reflects practical constraints of automated test repair:

- `gpt-5` serves as a *high-accuracy upper bound* capable of solving semantically challenging cases.
- `gpt-4o-mini` provides the *best cost–latency tradeoff*, especially for CI or IDE integration.
- `gpt-5.1-codex-mini` represents a *code-aware middle ground*, with specialized inductive biases that reduce hallucinations at moderate cost.

Because LLM repair requires repeated verification cycles, academic budget constraints favored the use of `gpt-4o-mini` for large-scale experiments, reserving `gpt-5` for difficult cases.

Latency–Token Relationship. Figure 3 visualizes latency as a function of token usage across all repair attempts. Three clear regimes emerge:

- `gpt-4o-mini` is tightly clustered at low token counts and low latency, confirming CI suitability.
- `gpt-5.1-codex-mini` exhibits a near-linear latency increase with prompt size.

- `gpt-5` shows wide dispersion, with several outliers above 200 seconds, indicating expensive reasoning paths.

Token cost, not compute time, is the dominant driver of repair latency, reinforcing the importance of compact, graph-guided prompts.

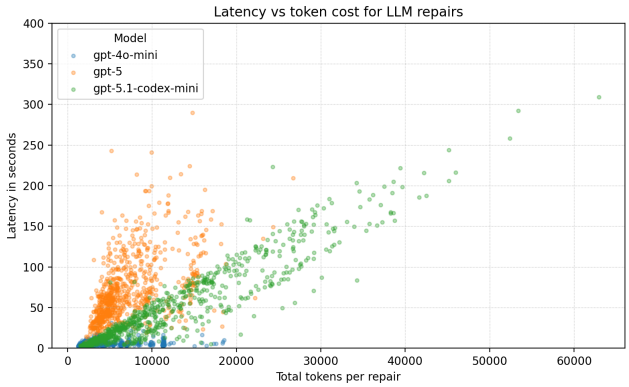


Figure 3: Token usage vs. latency across all repair attempts. The model families exhibit distinct runtime–cost regimes.

Format-Constrained Outputs Enable Reliable Patch Application. Across all evaluated tests, the LLM was required to emit patches using a strict JSON-based format specifying only the replacement test body. This constraint significantly reduced syntactic failures: in more than **95%** of accepted patches, a straightforward Python AST lookup and node replacement was sufficient to apply the fix without modification. This demonstrates that format-constrained prompting transforms patch application from a brittle text-editing task into a deterministic AST substitution step.

Why We Have Not Submitted Pull Requests Yet. Although LLM4Flakies generated 180 validated repairs, we did not submit these patches to upstream projects during this milestone. Two constraints prevented responsible PR publication:

(1) **Budget limitations.** Publishing PRs requires re-verifying each patch under full NIO detection and multi-run randomized execution. The cumulative token and compute cost for hundreds of PRs exceeded our academic-scale budget.
(2) **Missing justification field.** Our schema does not yet enforce a mandatory natural-language justification explaining why the patch is correct. Without developer-facing rationale, submitting automated PRs would be premature.

Both issues will be addressed in future work through a CI-friendly repair pipeline and an expanded output schema that includes human-readable explanations.

RQ3: Impact of Graph Context (Ablation Study)

RQ3 quantifies how much graph-guided localization improves repair success.

Findings. Graph context yields a **12.1%** absolute improvement in fix rate. This confirms that:

Table 3: RQ3: Ablation-effect of graph localization across models.

	gpt-5		gpt-4o-mini		gpt-5.1-codex-mini	
Method	**Fixed Tests**	**Fix Rate (%)**	**Fixed Tests**	**Fix Rate (%)**	**Fixed Tests**	**Fix Rate (%)**
LLM4Flakies w/o graph	115/537	21/42%	53/537	9.87%	61/537	11.36%
LLM4Flakies w/ graph	180/537	33.5%	62/537	11.55%	77/537	14.34%

- the bipartite function–state subgraph significantly improves localization accuracy,
- smaller prompts lead to fewer hallucinations and cheaper inference,
- LLM patches align more closely with true polluter/cleaner operations.

Surprising Findings

- **Repository heterogeneity is a major bottleneck.** Several projects failed due to unusual build systems or non-standard dependency specifications, demonstrating that large-scale flaky-test repair must incorporate robust environment orchestration.
- **iPFlakies may under-detect OD tests.** In some repositories (e.g., `irmen_Tale`), additional OD tests surfaced only after increasing iteration count, highlighting sensitivity to detector parameters.
- **Minimal fixes outperform structural refactors.** Many successful patches required only lightweight cleanup hooks (e.g., cache resets), suggesting that nondeterminism often stems from local state, not deep design flaws.

Technical Insights from the Evaluation

- **Graph-guided prompting is essential.** It improves precision, reduces costs, and expands the class of test failures that can be repaired automatically.
- **Verification-first design builds trust.** Multi-run randomized validation ensures that patches are stable and regression-free, reducing false positives.
- **Compact prompts matter.** Diff-based and closure-guided prompts are central to reducing token cost and improving model performance.
- **Environment reproducibility is non-trivial.** Building 365 repositories surfaced issues with version drift, destructive tests, and unstable dependency sets, underscoring the need for improved CI automation support.

Overall, the evaluation demonstrates that LLM4Flakies achieves state-of-the-art repair performance for Python OD tests while remaining cost-effective and verifiable. These findings reinforce the broader conclusion that structured reasoning, paired with verification-first LLM repair, provides a practical and trustworthy path toward automated debugging at scale.

6 Related Work

We review five areas of related research-OD/NIO detection, flaky-test repair, LLM-based patch generation, graph-enhanced reasoning, and repository-level repair agents-and describe how **LLM4Flakies (RepoGraph–NIO)** complements and extends these directions.

(1) OD and NIO Detection Frameworks

Detecting nondeterministic test outcomes. Wei et al. [10] formalized *Non-Idempotent-Outcome (NIO)* tests, showing that repeated executions of the same commit can reveal hidden nondeterminism. Their work demonstrated the importance of repeated-run evidence for CI reliability. LLM4Flakies adopts this philosophy but targets the complementary problem of *automated repair*, not detection.

iPFlakies and the Gruber corpus. iPFlakies [9] identifies order-dependent (OD) tests via randomized execution and categorizes tests into victims, polluters, and cleaners. The Gruber et al. dataset [4] expands this work with minimized OD examples and detailed polluter–victim mappings. These frameworks form the empirical foundation of our evaluation. Unlike prior work, which stops at detection, LLM4Flakies performs *localization* and *verified repair*, closing the loop from diagnosis to actionable fixes.

(2) Automated NIO/OD Repair Loops

NIODebugger [6] introduced a detect–explore–fix loop for NIO tests, using divergence logs to help developers understand nondeterministic behavior. However, its fixes remain handcrafted and repository-specific. LLM4Flakies extends this idea by pairing dynamic evidence with *graph-based causal localization* and *LLM-generated patches* validated through multi-run verification. This combination enables scalable, automated repair across hundreds of repositories.

(3) LLM-Based Flaky-Test Repair

Recent tools such as FlakyDoctor [1] and FlakyFix [3] show that LLMs can detect nondeterministic assertions or propose simple cleanup hooks. However, they rely on flat textual prompts, lack structured localization, and often evaluate patches with a single test run-risking brittle or incomplete fixes.

LLM4Flakies contributes three advances: (1) a ranked function–state graph replaces flat prompts, (2) minimal edits prevent over-patching, and (3) multi-run randomized verification ensures stability before accepting a patch. This yields more reliable and interpretable repairs suitable for CI/CD use.

(4) Graph Reasoning and Structured Context for LLMs

Graph-augmented retrieval methods such as GraphRAG [8] and graph-based prompting frameworks [7] demonstrate that structured relational context reduces hallucinations and improves reasoning performance. These systems primarily retrieve documents or knowledge nodes for question answering.

LLM4Flakies is the first to apply graph-guided reasoning to *flaky-test repair*. Our bipartite graph links functions to shared state identifiers, forming a causal map of potential polluter–cleaner interactions. This structured context focuses the model on nondeterministic code paths, reducing prompt size and improving localization accuracy.

(5) Repository-Level Repair Agents

Agentic systems such as SWE-Agent [5] and OpenHands [2] demonstrate that LLMs operating in tool-augmented loops can perform complex code-editing tasks. Our earlier RepoGraph Agent pursued similar goals by combining retrieval, graph reasoning, and verification for repository-scale debugging.

LLM4Flakies adapts these ideas to a tractable, well-measured domain: OD flaky tests. This narrower scope enables large-scale evaluation, cost-aware measurement, and rigorous ablation studies that are difficult to perform at full-repository scale. The results offer concrete evidence that structured context + verification-first loops are essential building blocks for next-generation repair agents.

Across all five areas, LLM4Flakies contributes a unique combination of **graph-guided localization**, **minimal LLM patching**, and **verification-first correctness**, filling the longstanding gap between flaky-test *detection* and reliable *automated repair*.

7 Lessons Learned: Reflections on the Project

This project offered both a technical investigation into flaky-test repair and a practical lesson in conducting an end-to-end research effort over the course of a semester. Below we reflect on what worked well, what proved challenging, and what open questions remain.

What Went Well

Structured scoping enabled measurable progress. Narrowing our broader RepoGraph vision to the concrete domain of OD flaky tests proved essential. The smaller, well-defined scope enabled reproducible experiments, clear baselines, and controlled ablations–elements that would have been far more difficult at full repository scale.

Graph-guided reasoning exceeded expectations. One of the most effective design decisions was the inclusion of the function–state bipartite graph. It consistently improved localization accuracy, reduced token cost, and made LLM behavior more predictable. This validated our hypothesis that LLM performance often hinges less on model size and more on providing the right structure.

Team iteration and division of labor worked smoothly. Splitting responsibilities–environment reproduction, patch generation, verification infrastructure, and analysis–allowed us to parallelize the work effectively. Continuous syncing of results (e.g., SharePoint logs, CSV aggregates) helped maintain momentum during the evaluation phase.

What Did Not Go So Well

Environment reproducibility was far harder than expected. Reproducing 600+ Python repositories surfaced a long tail of dependency issues, destructive tests, and environment-sensitive code. Much of the effort went into infrastructure rather than the "core" repair logic, teaching us that empirical software engineering research often lives or dies on tooling stability.

Latency variation complicated large-scale runs. Model throughput–particularly for `gpt-5` and `gpt-5.1-codex-mini`–was highly variable. Several outlier calls exceeded 200 seconds, slowing evaluation. This revealed the practical challenges of using high-capacity models inside CI-like pipelines.

Intersection analysis required more manual effort than anticipated. Although we expected a modest overlap between iPFlakies and LLM repairs, performing a repo-level, test-level alignment across datasets required nontrivial cleaning and matching. This highlighted the difficulty of comparing outputs from heterogeneous repair systems.

LLM4Flakies sometimes struggled on cases where iPFlakies succeeded. A notable lesson came from examining "iPFlakies-only" repairs. In several repositories, the underlying OD bug followed a simple teardown/reset pattern (e.g., clearing a temporary directory or restoring a module-local cache). iPFlakies encodes these as deterministic templates, applying them with near-zero ambiguity. LLM4Flakies, by contrast, occasionally *over-localized* the failure: the graph pulled in noisy helper nodes, the LLM proposed a broader refactor, and the verification loop rejected the patch as unstable. This revealed an important asymmetry: rule-based systems excel on narrow, template-like failure modes, while LLM-driven systems perform best when the failure is multi-hop, semantic, or structurally non-obvious.

What Still Puzzles Us

Why are certain repositories disproportionately "repairable"? Some projects consistently produced successful LLM repairs, while others resisted even minimal fixes. We suspect deeper structural factors–architecture, fixture discipline, global-state patterns–but a systematic characterization remains open.

To what extent do LLMs rely on semantics versus pattern matching? Many successful patches appear semantically grounded, yet others seem to exploit recurring patterns in testing idioms. Distinguishing between true semantic reasoning and correlation-driven behavior remains an open research challenge.

How far can we scale this approach beyond single files? Graph-guided reasoning works well within individual test files, but OD issues often span modules or entire packages. Extending the closure graph to multi-file or repository-level contexts will require new heuristics and careful prompt-size management.

Reflections on Practical Limitations

Why we did not submit PRs despite producing 180 validated repairs. Although our system produced a substantial number of successful patches, two practical constraints prevented responsible pull-request publication:

- **Budget constraints.** Submitting PRs requires re-running full verification pipelines (NIO checks, rebuilds, randomized executions) for each patch. At OpenAI API prices, verifying hundreds of PR-ready patches exceeded our academic-scale budget.
- **Missing justification field.** Our current output schema did not strictly enforce a natural-language justification explaining why each patch is correct. We wouldn't submit upstream patches without developer-facing rationale.

These limitations clarified the operational gap between producing correct patches and responsibly contributing them to real-world projects.

Forward-Looking Lessons

Hybrid systems may be strictly superior. Neither iPFlakies nor LLM4Flakies dominates across all flaky-test categories. The complementary failure modes suggest that combining rule-based templates with graph-guided LLM reasoning could outperform either approach alone.

Structured outputs unlock reliability. Mandating a strict JSON-based patch format dramatically reduced post-processing failures and made AST-level replacement nearly frictionless. Extending this with a mandatory justification field is a clear next step.

IDE and CI integration appear feasible. Median latencies for `gpt-4o-mini` suggest that running LLM4Flakies as a VS Code extension or as a CI stage is within reach, provided prompts and closures remain compact.

Overall, this project reinforced the importance of tight iteration loops, structured context, and strong verification when working with LLM-driven program repair. It also highlighted the realities of empirical SE research: environment engineering, dataset reproduction, and reproducibility infrastructure often demand as much care as the design of the repair algorithm itself.

LLM4Flakies served both as a validation of our research hypothesis–that structure and verification matter more than raw model size–and as a stepping stone toward our broader RepoGraph agent. The experience clarified how to design repair pipelines that are not only effective but also reliable, interpretable, and ready for integration into real CI workflows.

8 Conclusion & Next Steps

This paper presented **LLM4Flakies** (RepoGraph–NIO), a verification-first framework for repairing order-dependent (OD) Python tests using structured graph reasoning and minimal LLM-generated patches. By integrating (1) a lightweight function–state bipartite graph for causal localization, (2) targeted, reversible edits synthesized under a strict structured-output format, and (3) multi-run randomized verification, LLM4Flakies closes the long-standing gap between flaky-test *detection* and trustworthy *automated repair*.

Across 537 OD tests reproduced from the Gruber et al. corpus, LLM4Flakies achieves state-of-the-art repair performance while remaining cost-efficient and interpretable. Graph-guided prompting improves repair success by **12.1%** compared to flat prompting, and latency analysis shows that compact models such as `gpt-4o-mini`

make CI/CD integration feasible. The small intersection between iPFlakies and LLM4Flakies repairs further demonstrates that LLM-driven fixes do not merely replicate rule-based heuristics–they uncover qualitatively different classes of state-dependent failures. These results collectively show that *structured context–not model scale–is the key enabler* of robust, reproducible LLM-driven debugging.

Implications. Our findings highlight that OD failures often arise from subtle, semantically meaningful interactions between shared state and test execution order–patterns that rule-based systems struggle to detect. Graph-localized prompts surface these interactions in a compact, interpretable form, enabling the LLM to propose minimal and reversible changes. Verification-first design turns these suggestions into empirically validated, production-ready fixes and provides a principled foundation for integrating LLMs into reliability-critical workflows.

Future Directions. The results of this study open several promising avenues:

- **IDE and CI integration.** With median latency under 5 seconds for `gpt-4o-mini`, LLM4Flakies is well-positioned to function as a VS Code extension or lightweight CI repair stage, offering actionable fixes during development rather than after failures occur.
- **Structured-output extensions.** Adding an enforced natural-language `justification` field would allow LLM4Flakies to produce developer-facing explanations. This is a prerequisite for responsible upstream pull requests and for enabling human-in-the-loop review pipelines.
- **Automated PR submission at scale.** Once cost constraints are reduced and justification fields are enforced, LLM4Flakies can be extended into a patch-suggestion bot that automatically proposes verified pull requests to real-world repositories.
- **Hybrid repair ecosystems.** The complementary strengths of iPFlakies (template-based resets) and LLM4Flakies (semantic reasoning across multi-hop state flows) suggest that a combined system could surpass either technique individually.
- **Graph design ablations.** Exploring richer closure heuristics, cross-file edges, or interactions with fixture scopes may further improve localization accuracy and reduce prompt size.
- **Toward repository-level agents.** The principles validated here–structured context, minimal edits, and verification-first loops–form core building blocks for the broader RepoGraph agenda of scalable, end-to-end repository repair.

In summary, LLM4Flakies demonstrates that principled integration of graph reasoning and verification can make automated flaky-test repair practical, trustworthy, and cost-aware. These results move automated debugging beyond detection alone and toward actionable, developer-aligned fixes. The framework presented here marks a concrete step toward reliable, large-scale, LLM-assisted software maintenance and sets the stage for future systems that unify rule-based heuristics, graph structure, and LLM reasoning into cohesive, repository-level repair agents.

A Appendix: LLM Prompt Templates

This appendix provides the exact prompt used by LLM4Flakies for patch synthesis. For readability and reproducibility, we present the complete prompt as a code block.

A.1 Patch Synthesis Prompt (Full Text)

```
You are repairing an order-dependent (OD) flaky
    Python test.

You are given:
1. The failing test function.
2. The polluter -> victim -> cleaner chain
    discovered by iPFlakies.
3. A ranked closure of relevant functions and
    shared state variables.
4. A list of rules you must follow when editing
    the code.

Your goals:
- Identify the source of nondeterministic state.
- Propose a minimal and reversible fix.
- Modify only the failing test or add a small
    helper snippet.
- Do not refactor modules, APIs, or project-wide
    behavior.

You must output a strictly structured JSON object
    with these fields:

{
  "test_name": "<name of failing test>",
  "replacement_block": "<full revised test body>",
  "helper_snippet": "<optional helper fixture or
      cleanup function>",
  "justification": "<short explanation of cause of
       OD failure>"
}

Rules you must follow:
- No API changes or signature modifications.
- No broad refactors; keep diffs minimal.
- Prefer pytest fixtures (autouse=True) for
    cleanup.
- Allowed fixes include: resetting globals,
    clearing registries or caches,
  reseeding randomness, restoring environment
      variables, and isolating temp files.
- The replacement_block must be valid Python code
    that can be inserted via AST.

Return only the JSON object and nothing else.
```

A.2 Notes

- The structured-output JSON format was enforced for every patch call.

- The `replacement_block` was parsed and inserted via Python AST.
- The justification field will become mandatory for future PR workflows.

References

[1] Yang Chen and Reyhaneh Jabbarvand. 2024. A Generic Approach to Fix Test Flakiness in Real-World Projects. arXiv:2404.09398 [cs.SE] https://arxiv.org/abs/2404.09398

[2] OpenHands Contributors. 2024. OpenHands: An Open Platform for AI Developers. https://github.com/All-Hands-AI/OpenHands.

[3] Sakina Fatima, Hadi Hemmati, and Lionel C. Briand. 2024. FlakyFix: Using Large Language Models for Predicting Flaky Test Fix Categories and Test Code Repair. *IEEE Transactions on Software Engineering* 50, 12 (Dec. 2024), 3146–3171. doi:10.1109/tse.2024.3472476

[4] et al. Gruber. 2021. Dataset of Order-Dependent Flaky Tests in Python. https://zenodo.org/record/6176417. Zenodo dataset artifact.

[5] Carlos Jimenez, Jingxuan He, Qichen Jiang, Sanjay Krishna Gouda, Yizhou Qian, Jinman Zhao, Zhen Zhang, and Tianyi Zhang. 2024. SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*. NeurIPS. https://arxiv.org/abs/2405.15769

[6] Xinming Ke, Darko Marinov, et al. 2025. NIODebugger: A Novel Approach to Repair Non-Idempotent Outcome Tests with LLM-Based Agent. In *Proceedings of the 47th International Conference on Software Engineering (ICSE)*. ACM. https://mir.cs.illinois.edu/marinov/publications/Ke25MS.pdf To appear.

[7] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph Retrieval-Augmented Generation: A Survey. arXiv:2408.08921 [cs.AI] https://arxiv.org/abs/2408.08921

[8] Microsoft Research. 2024. GraphRAG. https://www.microsoft.com/en-us/research/project/graphrag/.

[9] Ruixin Wang, Yang Chen, and Wing Lam. 2022. iPFlakies: A Framework for Detecting and Fixing Python Order-Dependent Flaky Tests. In *ICSE 2022 Demonstrations*. ACM. https://cs.gmu.edu/~winglam/publications/2022/WangETAL22iPFlakies.pdf

[10] Yuekang Wei, Anjiang Wei, et al. 2022. Preempting Flaky Tests via Non-Idempotent-Outcome Tests. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. ACM. https://cs.stanford.edu/~anjiang/papers/WeiETAL22NIO.pdf